Final Project

Industrial Automation
MECE4613

Reference to required files (including this document):
~courseworks: Final-Lab-Project/

Introduction

As you know by now, Industrial Automation (IA) is a multidisciplinary field that requires wearing many hats to automate tasks effectively. Depending on the nature of the task, a successful automation demands an overall understanding of the relevant industrial or scientific fields, as well as a full comprehension of what being automated as a big picture.

This requires a Systems Engineering, *SE*, approach, where a foundational understanding across multiple disciplines is essential, more so than deep expertise in a single field. For instance, designing, prototyping, building, and operating a space shuttle involves knowledge of mechanical, control, electrical, chemical, and computer engineering, as well as material science, physics, and biology. Systems Engineers play a critical role in leading the entire design, prototype, and operational phases of the project, while they do not need to be experts in all areas. Imagine this role as akin to someone who is able to speak English, French, German, and Persian, but is neither Shakespeare, Hugo, Goethe, nor Rumi. Instead, Systems Engineers use a broad knowledge base to establish boundaries of responsibility and harmonize the various "languages" required to achieve the project's objectives efficiently and reliably.

Similarly, an Industrial Automation Engineer, *IAE*, is responsible for handling the entire automation project, bringing all the pieces together as a cohesive whole, rather than concentrating on one field at the expense of the others. This cohesion is achievable by managing how humans interact with the system, how data is collected and stored, how the model parameters should be trained and tuned, and how the various components of the system interact seamlessly with each other - all in an automated manner. Having a strong grasp of computer systems at the system level is crucial to make this cohesion possible.

In this final lab project, we will walk through a simple automation – yet fundamental tasks, focusing on some of the basics we learned. In part A, you will build a human machine interface (HMI) which is an important part of any automation task. In part B, your robot travels while detecting objects around. As soon as it detects your sign, it performs some functionality. In part C, there are a few IA- and AI-correlated questions you need to answer. The required knowledge and references to answer those question should be covered by the end of semester.

Part A – Building Industrial Automation HMI, 5 points

We'll build an HMI to control your robot via your local computer, tablet or a smart phone. For this purpose, we'll use Tornado web framework which you have already installed in your robot's computer. Open `hmi1.py` script to see how the different parts work. The listening port is defined tentatively PORT=8888 as a channel to talk to your robot. DEBUG is going to store any error or debug value which the robot may encounter. The html file is a theme file in which your web browser in your local computer, tablet or smart phone should load to provide users access to your hardware. The temple which this script uses is `hmi1.html`, which is located in your script's path. Handler class, should handle all the external requests. The `get(self)` method, first makes "stamp" variable, then renders your html file, sending your variable to the page. First let's take a look in the a/m html file:

```html
<!DOCTYPE HTML>
<html lang="en">
<head>
  <title>IA HMI</title>
  <meta name="viewport" content="width=device-width">
  <link rel="icon" href="data:,">
<style>
body {
  font-family: Verdana, sans-serif;
}
</style>
</head>
<body>

<h1>IA Lab</h1>
<h2>Robot HMI</h2>
<h3>John Doe, jdxxxx</h3>
{{ stamp }}

<form method="post" action="choice_a">
  <input type="submit" value="choice A">
</form>
<form method="post" action="choice_b">
  <input type="submit" value="choice B">
</form>

</body>
</html>
```

Page setup parameters

What should be displayed in the page; `stamp`: dynamic information to be retured from your machine.

Tells when choice X is pressed what action in the method `post()` has to be taken. These actions must be defined in the basic running machine.

Fig. 1 – Anatomy of hmi1.html file

Run `hmi1.py` on your machine (your robot's computer). This opens the port you've assigned in your machine to listen to the external requests. Now, in any remote computer or tablet or

smartphone, connected to your machines' network[1], open any browser and input the following address:

http://<your_machine's_ip_address>:<PORT>

If you run this successfully, you'll see a screen as following:

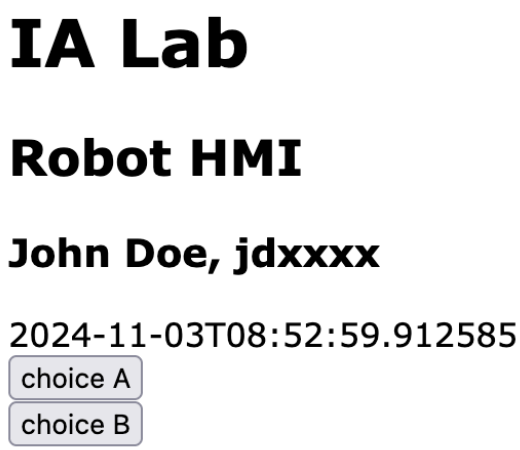


Fig. 2 – HMI primary connection test
hmi1.py and hmi1.html

At this first version of HMI, our machine returns only a time stamp to show to the user[2].

For the given `hmi.html` file, complete `hmi-ex.py` to perform machine's functions. If you run this on your machine's computer, and browse in your remote computer, you'll see this image:

---

[1] In many industrial scenarios, the machine and HMI are not connected to the same network. Then, an intranet or virtual private network (vpn) should be defined company-wide as so both the machine and HMI server, could connect to the same vpn – even though they are geographically apart.

[2] An IAE, can send back as many data readings as much it's needed. These include – but not limited to: sensors readings, positions, reinforcement learning agent's interactions with environment, machine learning models' outputs, etc. The collected information are normally categorized as "events". Refer to class lectures to understand event systems. One possible methodology of handling events is using WebSocket

Fig. 3 – HMI using a smartphone
hmi-ex.py and hmi.html

When you push the buttons, it won't do anything. To complete, replace pass with proper commands to operate the robot. You are given the full script and html file; this is literally 3 lines of codes in total and should be a 5 min job after completion of Assignment 2.v

Part B – Dynamic Object Detection, 40 points

In this part, each student's robot will detect a unique QR code (your university ID), stop for 3 seconds with a blinking LED, and then return to its starting station. This 3-second pause simulates a real-world opportunity to perform an action, like pushing or grabbing an object, highlighting the flexibility and reactivity of automation systems.

This embodies essential principles of Industrial Automation that are foundational to modern manufacturing, logistics, and many other industries. The skills and concepts you'll practice here - such as sensor-based detection, programmed responses, and timing control - are crucial for developing systems that respond autonomously to their environments.

When a robot identifies a QR code and performs an action, it's simulating a real-world task in automation, where machines need to detect objects, make decisions, and perform operations with precision. Many industries rely on such systems to streamline processes, enhance efficiency, and maintain consistency in tasks that would otherwise require significant manual effort.

Real-World Applications of QR Code Detection and Task Execution:

Let's explore a few industrial scenarios that are directly related to what you'll practice in this project:

1. **Automatic Inventory Management in Warehouses**:

   Robots equipped with sensors scan barcodes or QR codes on packages to identify them, ensuring the right products are sent to the right location. Your project's stop-and-detect function simulates this principle, where each QR code represents a unique product or package that the system must identify.

2. **Automated Inspection and Quality Control**:

   Industrial automation systems often stop briefly to scan or inspect an item. For instance, a camera might detect flaws on a product's surface. Similarly, your robot's pause and blink action is akin to these inspection pauses, teaching you the significance of a controlled and purposeful halt in automation.

3. **Automatic Sorting and Categorizing**:

   In facilities like recycling plants or logistics hubs, items are sorted by detection and categorization. For example, each QR code could represent a different material or destination. When detected, the robot could trigger different actions, such as moving items to specific bins. This project helps you see how detection-triggered actions are useful for sorting tasks.

4. **Autonomous Safety Checks and Obstacle Detection**:

   In hazardous environments, automated robots often stop when certain patterns or markers are detected to avoid potential risks or obstacles. This project simulates that kind of safety stop, emphasizing the role of automated safety mechanisms in industry.

Why the 3-Second Delay Matters:

The delay serves more than a pause; it teaches you that detection-triggered events in automation can enable complex actions, like interacting with items. For example, a robot might stop at a station to pick up, examine, or release an object. This intentional pause not only helps simulate realistic industrial processes but also encourages you to think creatively about what actions a robot could perform once a target is spotted and how to be physically linked to the object.

For this purpose, you may install `qrcode` in your robot's computer:

```
$ pip install qrcode -y
```

Now you can create your personal QR code. If your UNI is am3469, then:

```
$ qr am3469 > am3469.png
```

You may copy this file to your local computer to print it. Your robot will travel along a straight line where the codes are on the side. Your robot should stop on your code only – not on the others. If there are more than one code of your UNI, then it has to stop in all of them.

Note that the speed of the robot does not matter.

Part C – Questions, 25 points

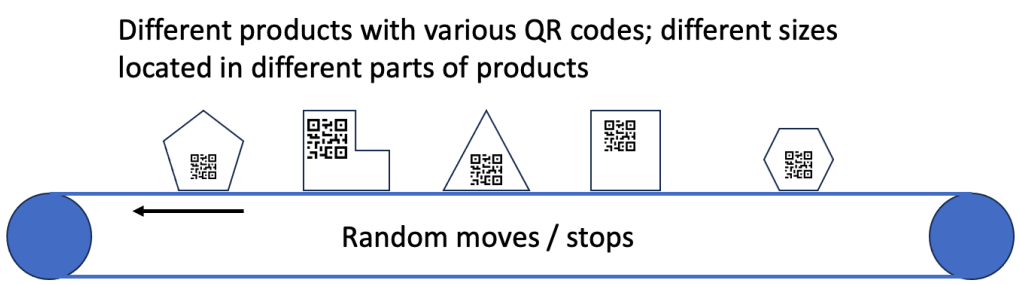
Work on the following questions and hand in by the due date.

NOTE:
Studying and using class lectures, will potentially change your writing's language. Then, your answers will be aligned with standard Industrial Automation terminologies, leading for a better score.

Question 1:
HMI is a round-trip information exchange.

a) List a complete machine's parameters which might be controlled via your HMI.
b) List the required readings that the user should observe when the machine operates.
c) List emergency stop scenarios and how the user should respond.
d) List security considerations and if they should be appeared in the interface / self-addressed?

Question 2:
Imagine a belt conveyor carrying different products each with their own barcode or QR code. The conveyor could stop / move anytime in either left or right directions. The robot which can move along the conveyor, receives a signal via webapp or an HMI to spot and push some products outside of the conveyor. What changes do we need to apply to our robot to be able to cope this task efficiently?



Question 3:

What components need to be added to your robot for a *quick* pushing? What control algorithms would you need to add for a better and *rapid* action? Imagine this as you have just 10 milliseconds to make a decision, come up with algorithm and push the assigned product. What basic considerations would you need for this scenario, what bottlenecks do you have and how are they different when the *speed of action* is not a concern?

Question 4:

Your robot has to serve two different parallel belt conveyors. What design consideration should you use for an optimum autonomy? What if the robot receives more than one requests to push out a product; and several products?

Question 5:

This "belt conveyor and robot" system – the field, is classified as environmentally dangerous since there is a nuclear radiation in the area. Human operators have to be at least 25 miles away. The company needs two HMIs for safety and redundancy purposes, with XOR[3] operational logic. Each HMI has to be registered with a server which is connected to the robot's computer. Clarify the communication architecture.

Question 6:

The QR code is on a box which carries a material that needs to be removed out of the box with ultimate care. The code, posts to a web page which has handling instruction including the size, mass and the force needs to be gripped by. Design this system that performs this task. Explain in detail the hardware, software and additional control system.

Question 7:

Some products (i.e., QR codes) are assigned "urgent" or critical to kick out of the conveyor. This means that if the product is not pushed, an action should be taken. Explain how we can address this and how it is going to impact our hardware / software.

Question 8:

For question 7, we want to record data for:

- the incoming requests
- the actions taken
- the results.

These data will be recorded in the server which communicates with the field. Explain:

- SCADA system and the parameters to be monitored
- Required HMI for this task.
- What has to be demonstrated for the last software layer.
- A holistic AI system which is correlated with each software layer. What models should we train and how. What data schemes does these fit into.

---

[3] In this condition, the robot won't operate if both HMIs are active. Meaning that only one of them has to be active (registered) to be able to control the robot – if needed at all.